Department of Engineering Science
School of Science & Technology



CES 440 Data Communications
Ch.3 The Data Link Layer

Dr. Jingxian Wu
(707) 664-3491
Jingxian.wu@sonoma.edu

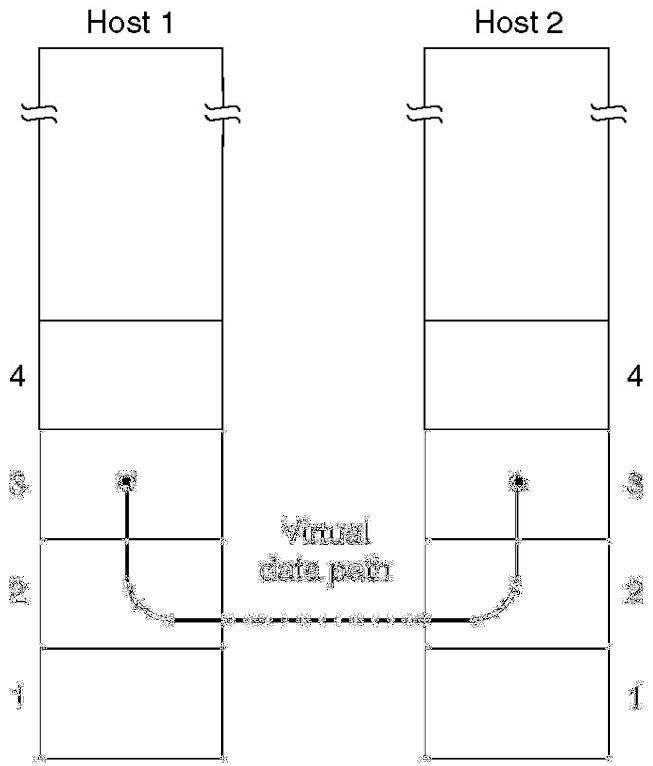
OUTLINE

- **Functions and Design Issues of Data Link Layer**
- **Error Detection and Correction**
- **Elementary Data Link Protocols**
- **Sliding Window Protocols**
- **Example Data Link Protocols**

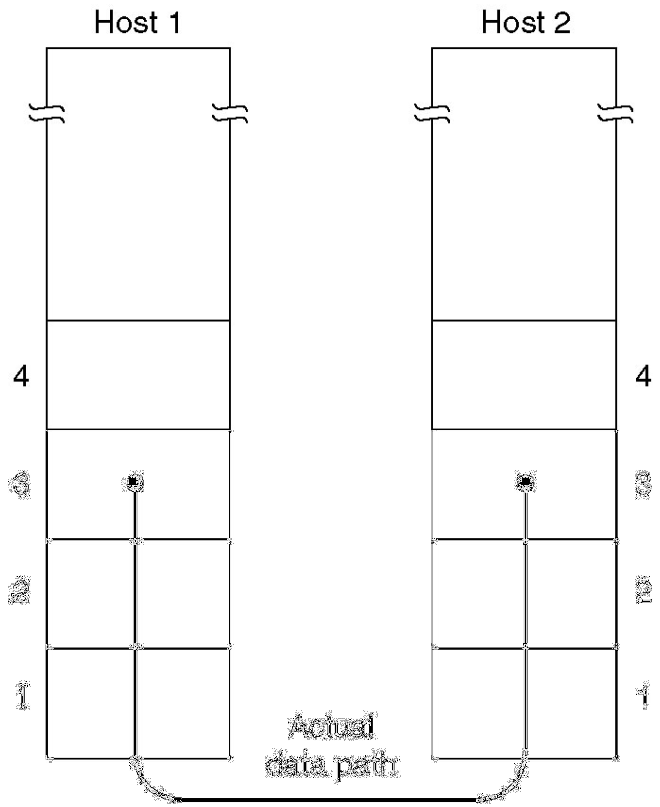
FUNCTIONS

- **Deals with reliable communication between two adjacent machines.**
  - The channel acts as a wire ➔ no switching or routing
- **Functions of data link layer**
  - Providing well-defined service interface to the network layer
    - Unacknowledged connectionless service
    - Acknowledged connectionless service
    - Acknowledged connection-oriented service
  - Framing
    - Encapsulates packets from network layer to frames.
  - Flow control
    - Keep fast transmitter from swamping slow receiver
  - Dealing with transmission errors
    - Error detection
    - Error correction

FUNCTIONS: SERVICES



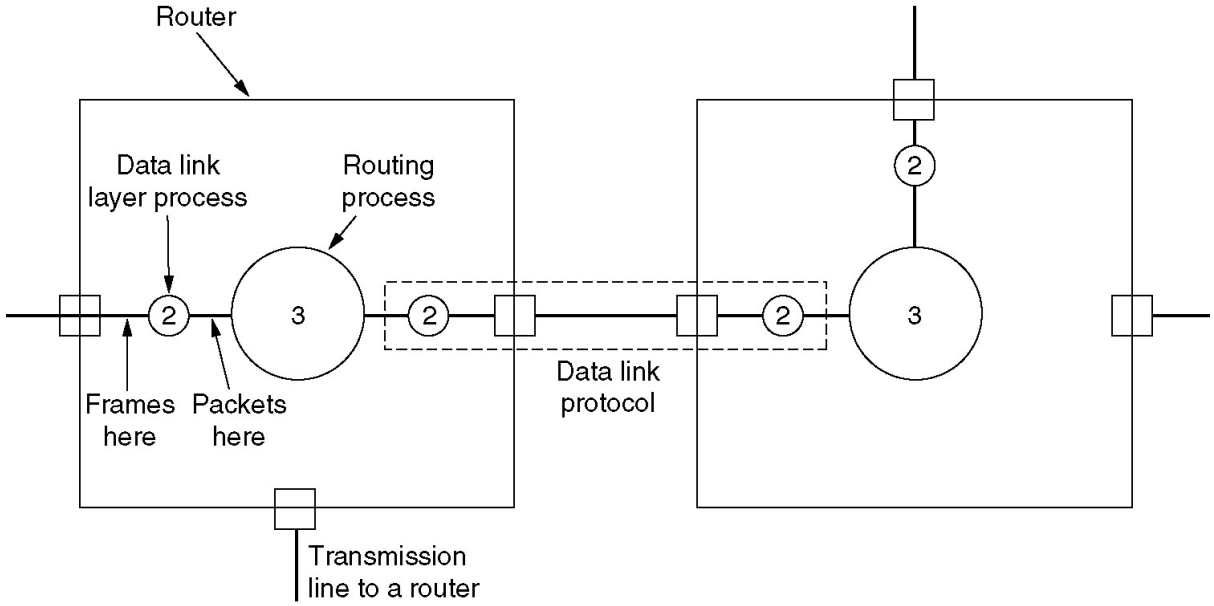
(a) Virtual communication



(b) Actual communication

FUNCTIONS: SERVICES

- **Routers are connected by point-to-point links**
  - E.g. leased lines
- **The entity of data link layer is usually a software**
  - Might be embedded in a chip
  - One router might have multiple different data link layers
    - E.g. connecting both wireless network and wired network

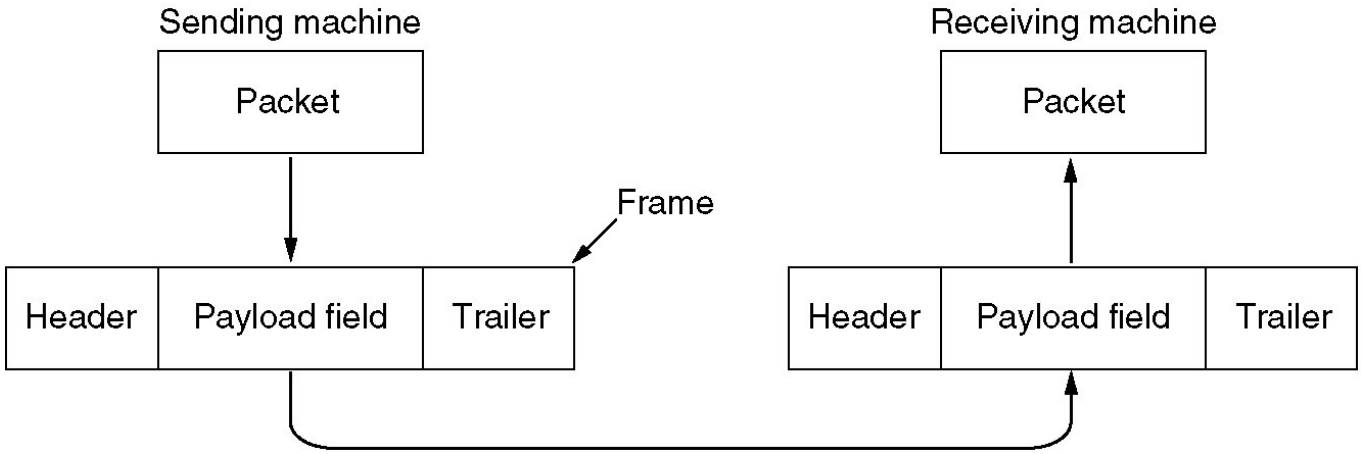


FUNCTIONS: SERVICES

- **Unacknowledged connectionless service**
  - No logical connection, no acknowledgement
  - Suitable for system with very low error rate
    - Leave error recovery to upper layer
  - Most LANs, such as Ethernet, are using this service in data link layer.
- **Acknowledged connectionless service**
  - No logical connection, but each frame is acknowledged.
  - Suitable for unreliable channels, such as wireless channel.
- **Acknowledged connection-oriented service**
  - Requires logical connection
    - Three phases: establishment, data transfer, termination
  - Each frame is acknowledged.
  - Example: LAPB (Link Access Procedure Balanced)
    - 2nd layer of X.25

FUNCTIONS: FRAMING

- **Framing**
  - Encapsulate packet from network layer to frame
    - Breaks down larger packets
    - Add header and trailer

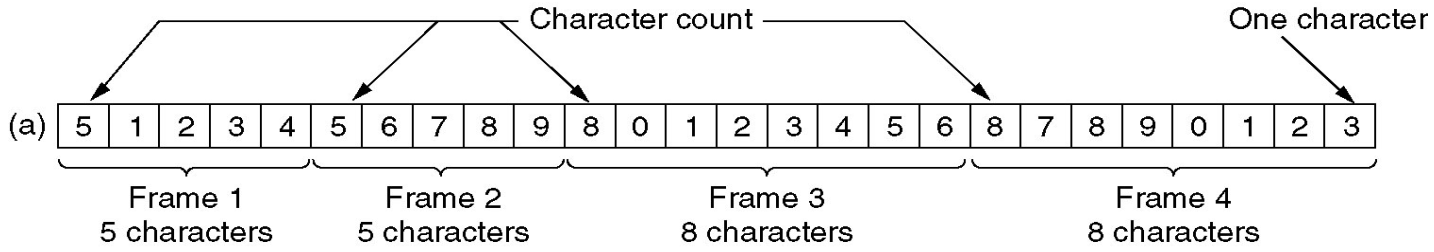


  - Break raw bit streams from physical layer into streams
    - Process each stream individually

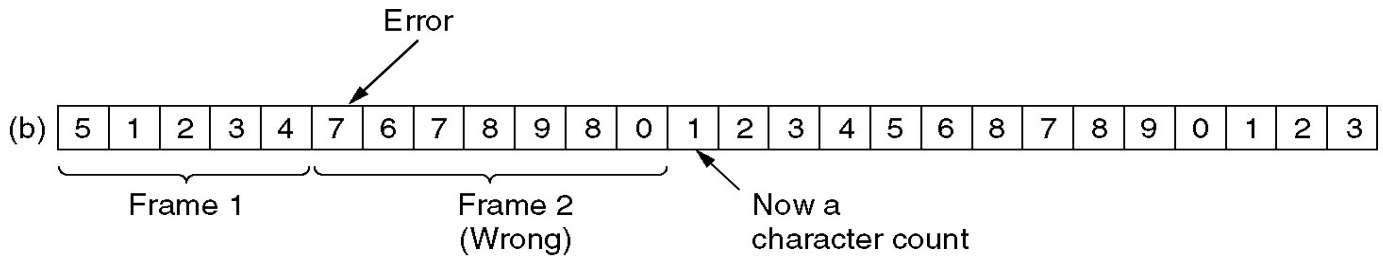
FUNCTIONS: FRAMING

- **Framing method # 1: Character Count**
  - A field in the header specifies number of bits in frame
  - Problem: what if an error occurs to the number of bits field in header?
    - Synchronization lost, following frames not recognized

No Error



One Error

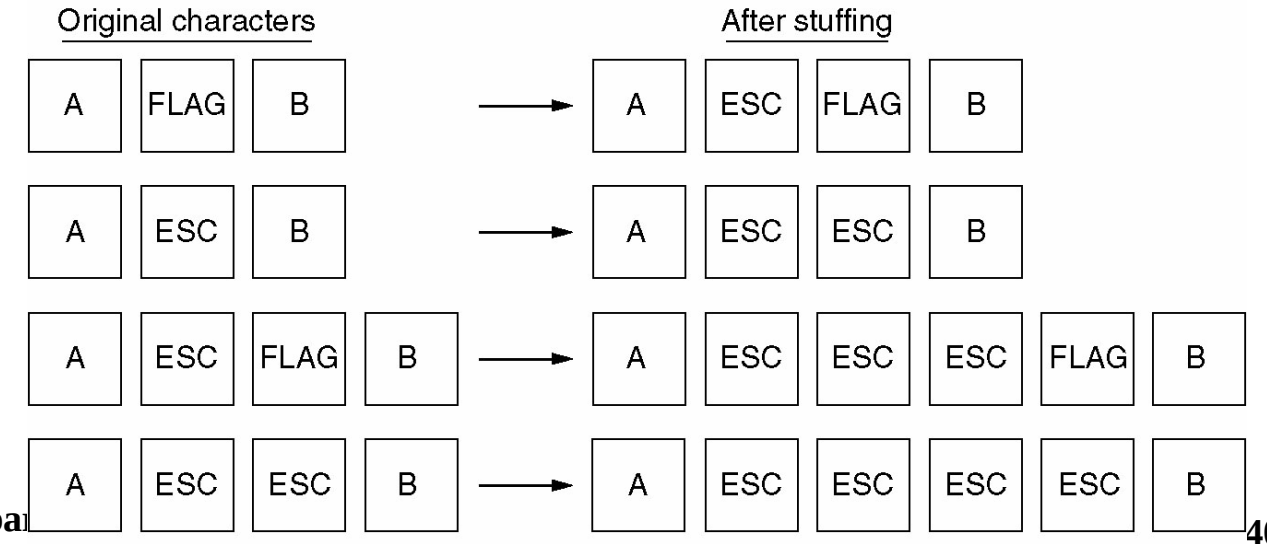


FUNCTIONS: FRAMING

- **Framing method # 2: Flag byte with byte stuffing**
  - Each frame starts and ends with special characters.
    - The special character is called flag byte.

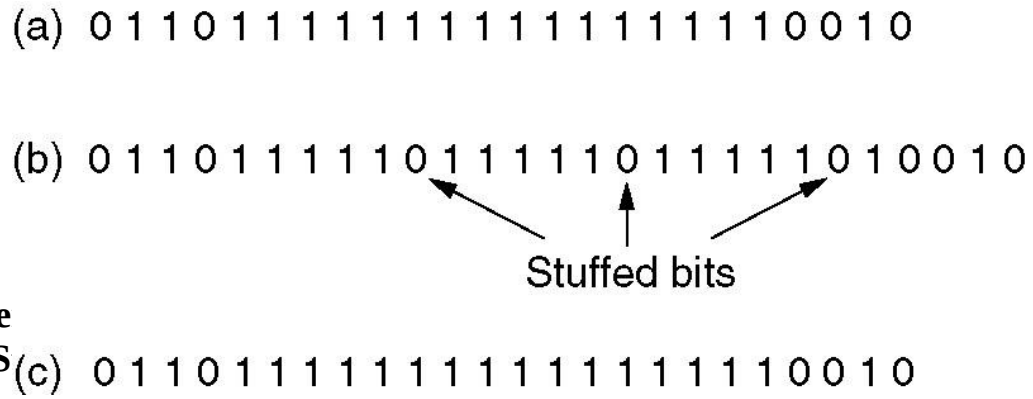


  - Problem: what if flag byte occurs in the middle of data
    - Solution: byte stuffing → Sender's data link layer inserts special escape byte (ESC) before each "false" flag byte and ESC

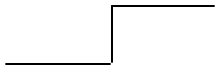
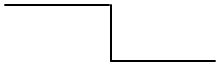


FUNCTIONS: FRAMING

- **Framing method # 3: Flag with bit stuffing**
  - Flag is a special bit pattern, and it can contain arbitrary number of bits
    - For flag byte, each flag must have 8 bits
  - Each frame starts and ends with a special bit pattern
    - E.g. 01111110
  - What if the Tx data contains this special bit pattern?
    - Solution: bit stuffing→ inserting bit to breakdown the flag bit pattern.
    - E.g. at the sender's data link layer, whenever there are 5 consecutive '1's, stuffs a '0' bit at the end of the stream
    - More efficient compared to byte stuffing



FUNCTIONS: FRAMING

- **Framing method # 4: Physical layer coding violation**
  - Use some impossible bit pattern to delimit frame
  - E.g. some LANs encode 1 bit of data by 2 physical bits
    - '1': high-low pair of voltage 
    - '0': low-high pair of voltage 
    - Thus high-high and low-low are impossible
      - They can be used to delimit frame
  - Only applicable to physical layer with special encoding schemes
    - Violate the design essence of layered structure!

- **In practice, many data link protocols use a combination of character count and one other framing methods for extra safety.**

FUNCTIONS: FLOW CONTROL & ERROR CONTROL

- **Flow Control**
  - Keep high speed Tx from swamping low speed Rx
  - Feedback-based flow control
    - Rx sends back information giving it permission to send more data.
  - Rate-based flow control
    - built-in mechanism limits the Tx rate of sender.
    - Never used in data-link layer. But used in upper layers.
- **Error Control**
  - Acknowledgement (Ack)
    - Positive Ack: Tx successful. Negative Ack: Tx unsuccessful.
    - Timer used at Tx➔no Ack before timer expired = negative Ack.
      - If Ack is lost, Rx will have multiple copies of same frame
      - Sequence # is used to ensure no duplication and loss.
  - Error detection and correction code.

OUTLINE

- Functions and Design Issues of Data Link Layer
- **Error Detection and Correction**
- Elementary Data Link Protocols
- Sliding Window Protocols
- Example Data Link Protocols

ERROR DETECTION AND CORRECTION

- **Basic idea**
  - Add redundancy in the original data to protect the transmitted data.
    - E.g. tx the same information several times (repetition code)
- **Two kinds of error control codes**
  - Error Correction
    - Include enough redundant information such that the Rx can recover the original information by looking at the Rx data.
      - E.g. repeat '1' 3 times. Tx (1 1 1), Rx (0 1 1) ➔ Receiver will guess that (1 1 1) is transmitted
  - Error Detection
    - Include only enough redundant information such that the Rx can detect an error by looking at the Rx data.
      - E.g. repeat '1' twice. Tx (1 1), Rx (0 1) ➔ Receiver knows there is an error, but couldn't guess what is transmitted
      - Send back Negative Ack (NAK).

ERROR DETECTION: PARITY CHECK

- **Parity Check**
  - Append a parity bit to the end of the data block
  - Original information along with parity is called codeword.
- **Even Parity**
  - The parity bit is chosen such that information and parity have even number of '1' in total.
  - E.g.: original information 101, parity bit should be '0' , the information Tx in the channel: 1010
  - The receiver simply counts the number of '1' in the received codeword.
    - If there are even number of '1', it assumes no error.
    - If there are odd number of '1', it assumes there is an error.
- **Odd Parity**
  - The parity bit is chosen such that information and parity have odd number of '1' in total.
- **Problem of 1-bit parity check**
  - If even number of error occurs, the Rx couldn't detect the error.
  - E.g. even parity check, Tx 101101000, Rx 111001000

ERROR CONTROL CODE: LINEAR BLOCK CODE

- **Linear Block Code**
  - m-bit of information, r-bit of parity check, codeword length (m+r) = n
  - Called (n,m)-code, code rate: m/n
  - Each parity bit is calculated based on part of the information bits
  - Binary operation

$$0 \oplus 0 = 0 \qquad 0 \oplus 1 = 1 \qquad 1 \oplus 0 = 1 \qquad 1 \oplus 1 = 0$$

  - Example (5, 3)-code:

Codeword: $\underbrace{b_1\ b_2\ b_3}_{\text{information}}\ \underbrace{p_1\ p_2}_{\text{parity}}$

Parity Check Calculation Example:

$$p_1 = b_1 \oplus b_2, \quad p_2 = b_1 \oplus b_2 \oplus b_3$$

Information 001 $\quad p_1 = 0 \oplus 0 = 0, \quad p_2 = (0 \oplus 0) \oplus 1 = 1$

Codeword 00101

- **Use n-bit to represent m-bit of information (n > m)**
  - More bits are Tx in channel ➔ larger data rate ➔ larger bandwidth
  - Tradeoff between bandwidth and system performance!

OUTLINE

- Functions and Design Issues of Data Link Layer
- Error Detection and Correction
- **Elementary Data Link Protocols**
- Sliding Window Protocols
- Example Data Link Protocols

ELEMENTARY PROTOCOLS

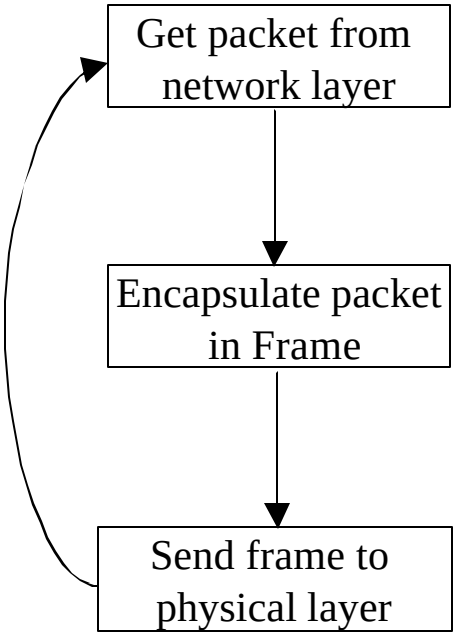
- **Assumptions**
  - Host A wants to send Host B an infinite long message
    - There is always message available in the network layer of A.
  - Hosts never crash.
  - Network layer passes packet to data link layer; data link layer encapsulates it in a frame by adding header and trailer.
    - Data link layer doesn't care the contents of packet.
  - The sender and receiver processes are dedicated completely to one link.
- **Event Driven**
  - Initially, the sender or receiver are in idle state (do nothing but waiting).
  - Once an event arrives, sender or receiver will process the event.
  - After the event processing, sender or receiver will go back to idle state.
  - Sample events:
    - frame_arrival, cksum_err, timeout, etc.
  - Practical system is usually interrupt driven.

ELEMENTARY PROTOCOLS

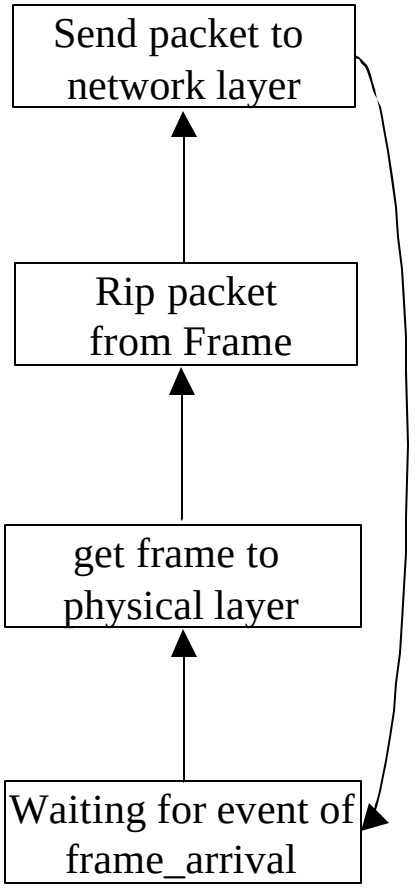
- **Service Interfaces**
  - to_network_layer(packet *s), from_network_layer(frame *s)
  - to_physical_layer(frame *s), from_physical_layer(frame *s)
  - enable_network_layer(void), disable_network_layer(void)
- **Event Interfaces**
  - void wait_for_event(event_type *event)
    - A while loop only returns when an event occurs
- **Timer Interfaces**
  - start_timer(seq_nr k), stop_timer(seq_nr k)
  - start_ack_timer(void), stop_ack_timer(void)
- **Data Structure & Constant**
  - packet, frame (including packet, frame_kind, seq#, ack#).
  - MAX_SEQ: frame # in the range of [0, MAX_SEQ].
- **Need to understand the programs in text book.**

PROTOCOL 1: IDEAL SIMPLEX

- **Sender**



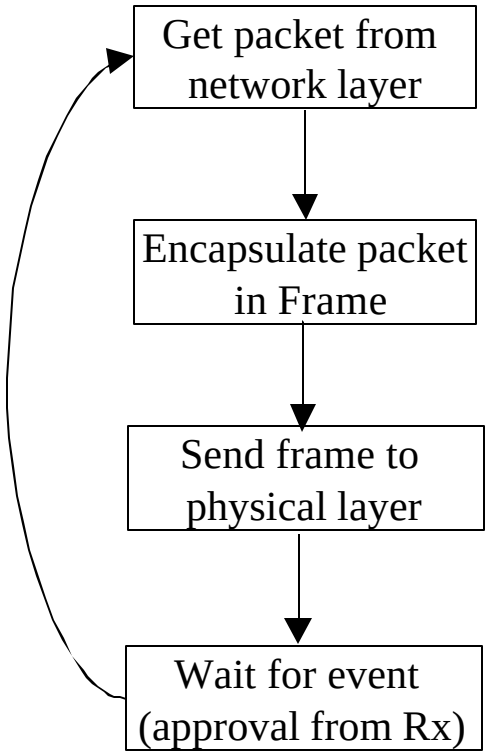
- **Receiver**



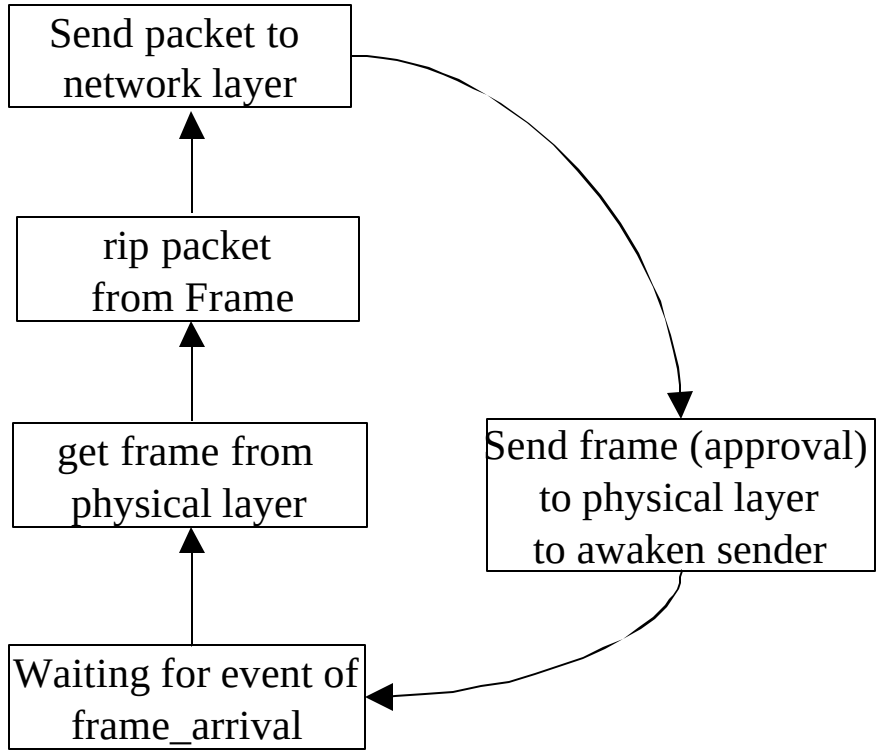
Ideal: no noise, no frame lost, no overflow

PROTOCOL 2: STOP-AND-WAIT

- **Sender**



- **Receiver**



Stop-and-wait: sender send one frame and then waits for an acknowledgement

- Simplest flow control, to avoid faster sender swamp slower receiver

PROTOCOL 3: AUTOMATIC REPEAT REQUEST (ARQ)

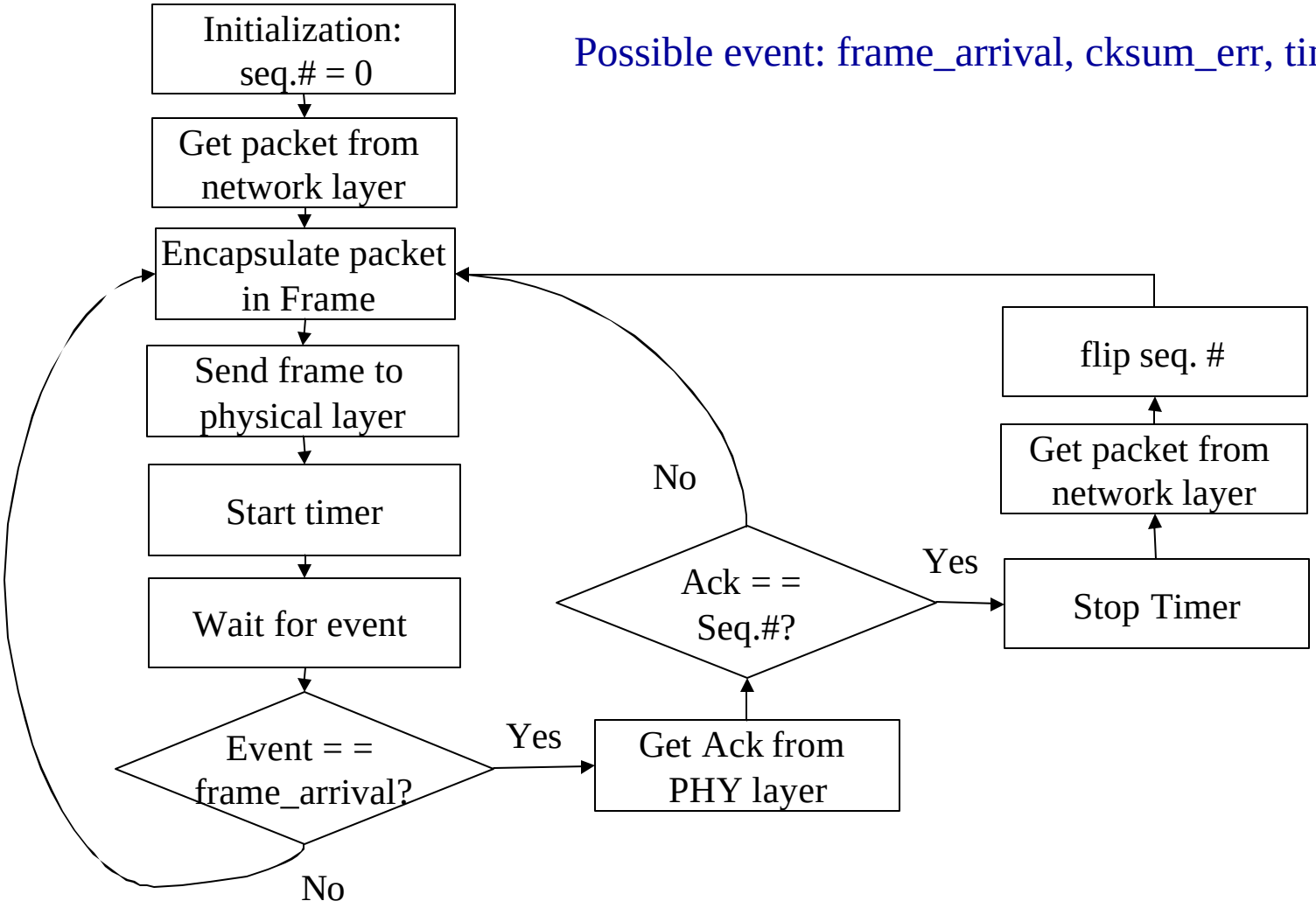
- **Noisy channel**
  - There might be errors in the Rx frame
  - The frames might get lost during transmission
    - Both data frames and Ack frame could get lost in channel
- **Problems and Solutions**
  - 1. Receiver receive the frame correctly
    - Sends back an acknowledgement (Ack)
  - 2. There is error in the Rx frame
    - Receiver does nothing.
    - Transmitter starts a timer after the frame is being sent out.
    - If the timer expires before Ack is received, sender re-Tx.
  - 3. The data frame get lost during tx.
    - Same as 2.
  - 4. The Ack gets lost during its way back to the sender.
    - Timer at sender expires. Sender re-Tx the same frame.
    - Rx is going to have two identical frames ➔ duplication
      - Associate each frame with a sequence# in the header to avoid duplication

PROTOCOL 3: ARQ

- **Sequence Number**
  - Part of the header information.
  - We want to use as less bitis as possible to represent sequence # ➔ less overhead.
- **What is the minimum # of bits needed for sequence # in ARQ?**
  - If the Ack for frame *m* is received by the sender
    - sender will send out frame *m+1*
  - If the Ack for frame *m* is lost in channel
    - Sender will retransmit frame *m*
  - The receiver only needs to distinguish between frame *m* and frame *m+1*.
  - 1-bit (0 or 1) information is sufficient to make the distinction in ARQ!
- **ARQ**
  - Sender waits for a positive Ack before advancing to the next frame.

PROTOCOL 3: ARQ

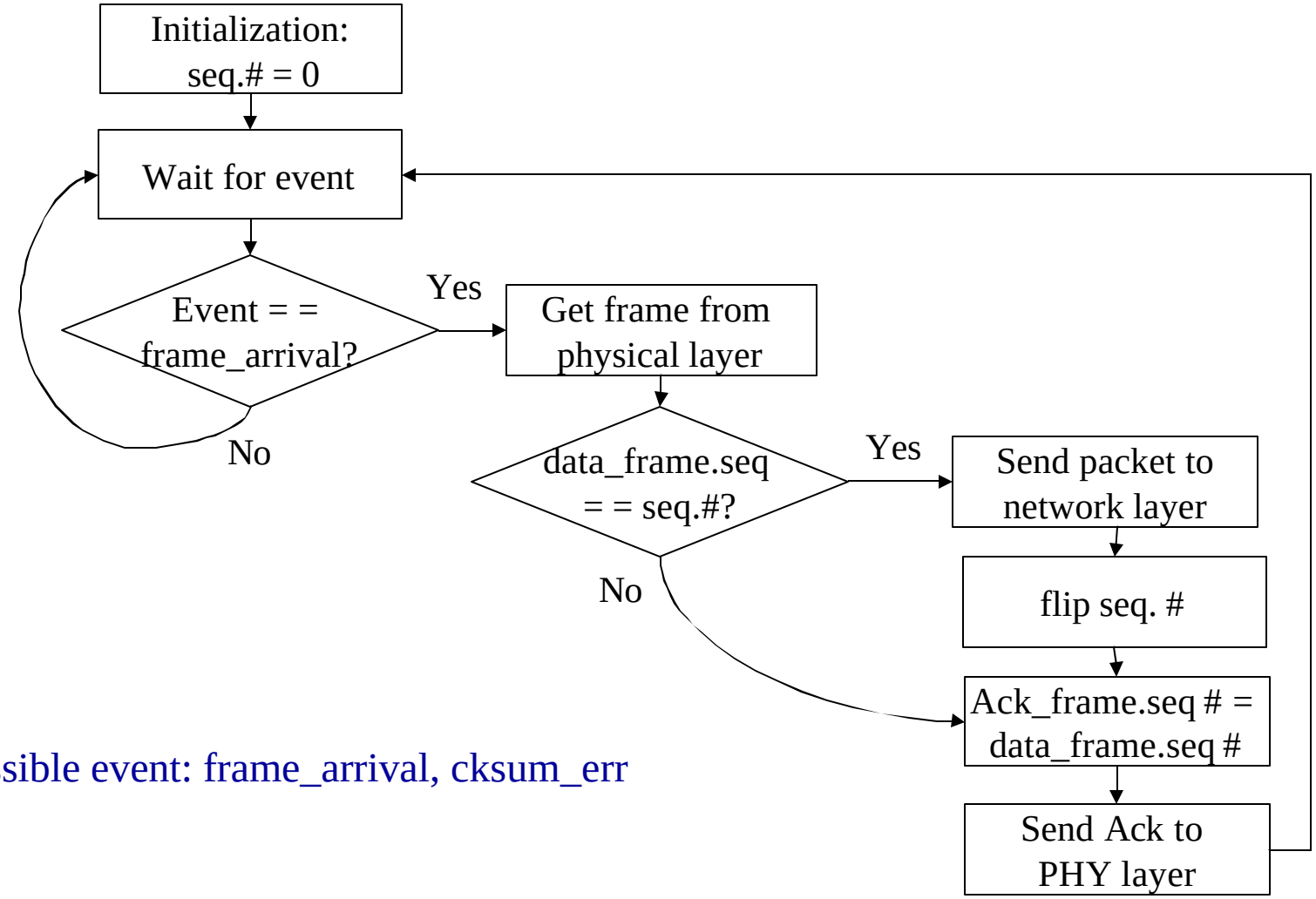
- **Sender**



Possible event: frame_arrival, cksum_err, time_out

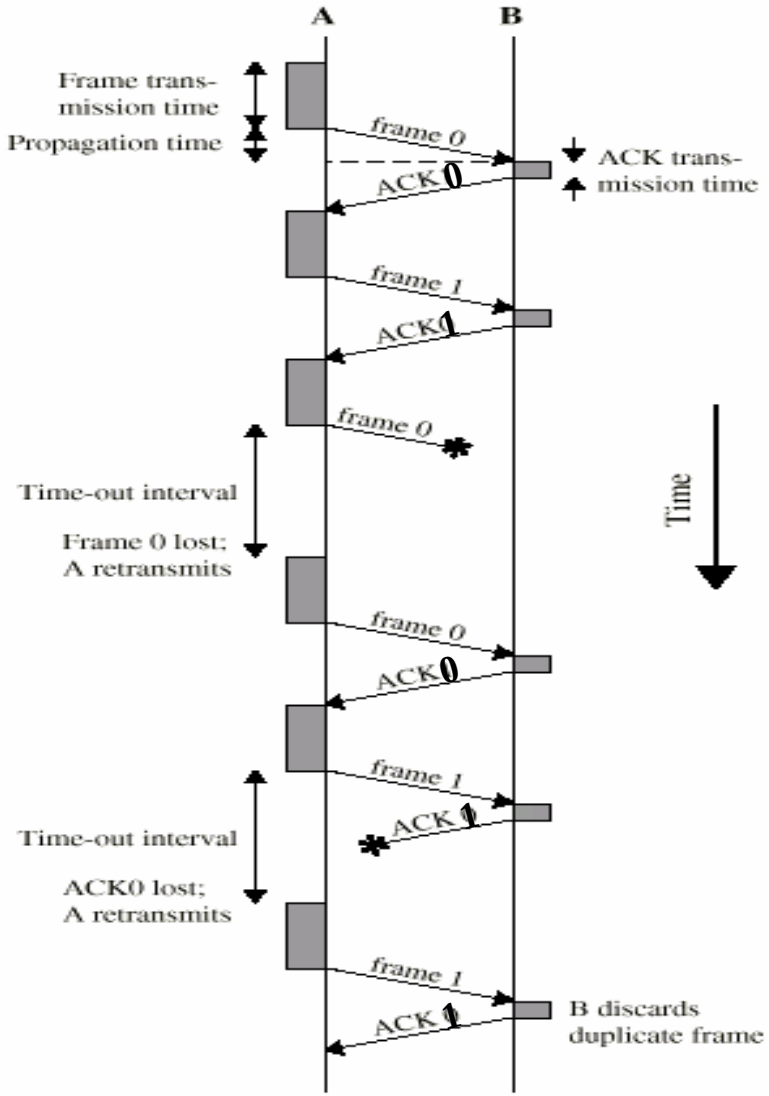
PROTOCOL 3: ARQ

- **Receiver**



Possible event: frame_arrival, cksum_err

PROTOCOL 3: ARQ



OUTLINE

- Functions and Design Issues of Data Link Layer
- Error Detection and Correction
- Elementary Data Link Protocols
- **Sliding Window Protocols**
- Example Data Link Protocols

SLIDING WINDOW: MOTIVATION

- **Motivation**
  - In stop-and-wait, the sender cannot Tx the next frame until positive Ack for the current frame is received.
  - If the round-trip delay is large, considerable time are spent on waiting.
  - What if we allow the sender Tx continuously without waiting for Ack?
    - Not good for flow control.
  - Solution
    - Allow the sender sends up to *m* frames without being Ack-ed

E.g. *m*=5

Sender sends out 0 1 2 3 4	(0 1 2 3 4 are not Ack-ed)
Sender receives the Ack for 1	(2 3 4 5 are not Ack-ed)
Sender sends out 6	(2 3 4 5 6 are not Ack-ed)
Sender receives the Ack for 4	(5 6 are not Ack-ed)
Sender sends out 7 8 9	(5 6 7 8 9 are not Ack-ed)

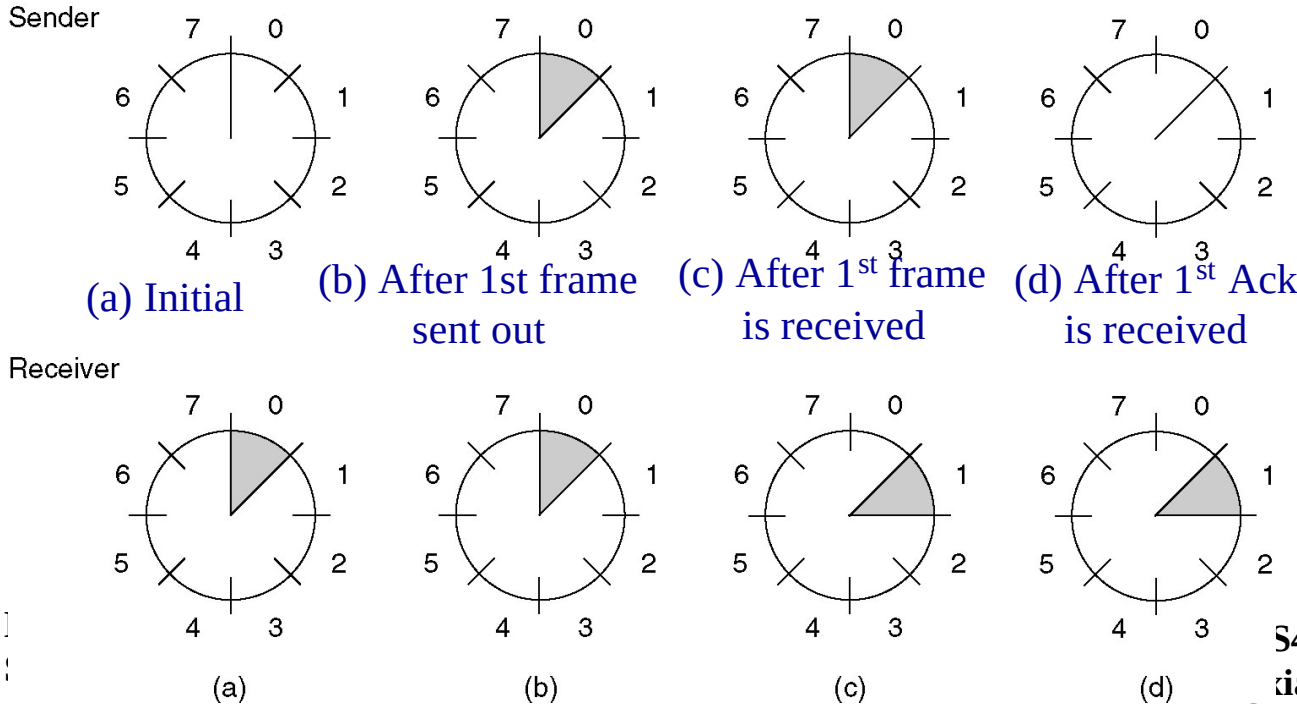
0 1 2 3 4 5 6 7 8 9

SLIDING WINDOW: HOW?

- **Sliding Window Protocol**
  - At any instant, the sender maintains a set of sequence numbers it's permitted to send ➔ sending window
  - At any instant, the receiver maintains a set of sequence numbers it's permitted to receive ➔ receiving window
  - Sending window and receiving window doesn't need to be of the same size
- **Sequence Number**
  - Each frame is associated with a sequence #
    - Maximum sequence # needs to be larger than window size
    - Can we use unlimited sequence #?
      - No. we want the maximum sequence # to be small to save BW.
  - Usually *k*-bit of the header is used to represent the sequence #
    - Range of sequence # : $[0, 2^k - 1]$
- **Stop-and-wait protocol is a special case of sliding window with *m* = 1 (window size) and *k* = 1 (the number of bits dedicated to sequence #).**

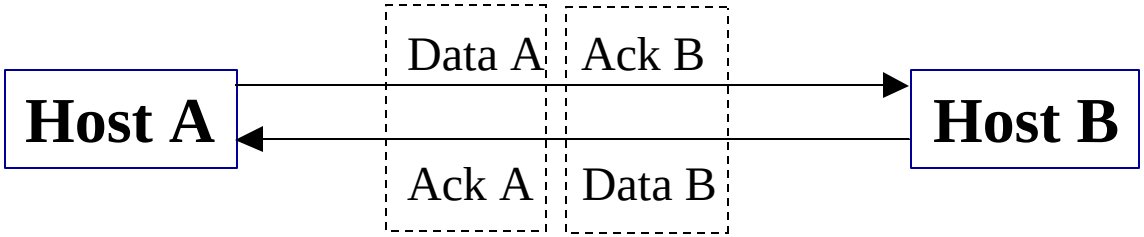
SLIDING WINDOW: HOW?

- **Example: *m* = 1, *k* = 3**
  - Range of sequence #: $[0, 2^3 - 1 = 7]$
  - Sender: when Ack comes in, the lower edge of the window advances 1.
  - Receiver: when a data frame with seq# equal to the window is received, the window is rotated by one.
    - Receiver's window always keeps the same size.



SLIDING WINDOW: FULL-DUPLEX

- **In previous examples, we assume one direction transmission**
  - Data frames are sent from host A to host B (forward channel)
  - Ack frames are sent from host B to host A (reverse channel)
- **Full-duplex transmission**
  - The physical channel from A to B
    - Data frames initiated from host A (forward channel)
    - Ack frames sent back to host B (reverse channel)
  - The physical channel from B to A
    - Data frames initiated from host A (forward channel)
    - Ack frames sent back to host B (reverse channel)



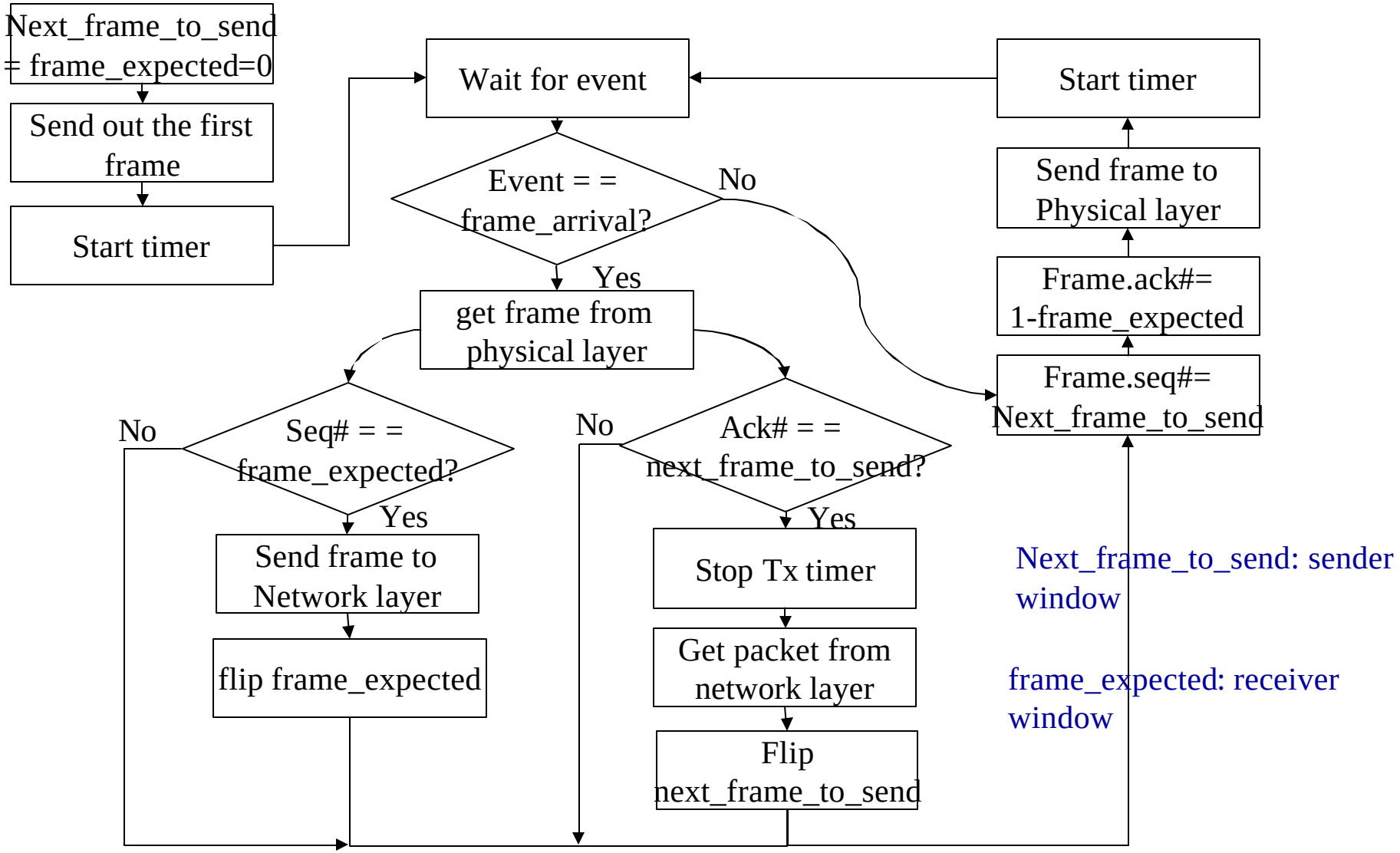
SLIDING WINDOW: FULL-DUPLEX

- **Piggyback**
  - Instead of sending out an individual Ack frame, embed the Ack information in the header of the next outgoing data frame.
- **Advantage: save bandwidth**
  - An individual Ack frame needs a header, a checksum, Ack information, and maybe more.
  - The piggybacked Ack information only contains several bits in the header of the next data frame.
- **Problems: what if there is no data frame to send?**
  - Solution: use a separate timer for acknowledgement at the receiver.
  - If the timer expires, and no data frame for the receiver to Tx, a separate Ack frame will be sent
  - The Ack timer must be significantly shorter than the re-Tx timer at sender.
    - Why?

SLIDING WINDOW PROTOCOL 1: 1-BIT WINDOW

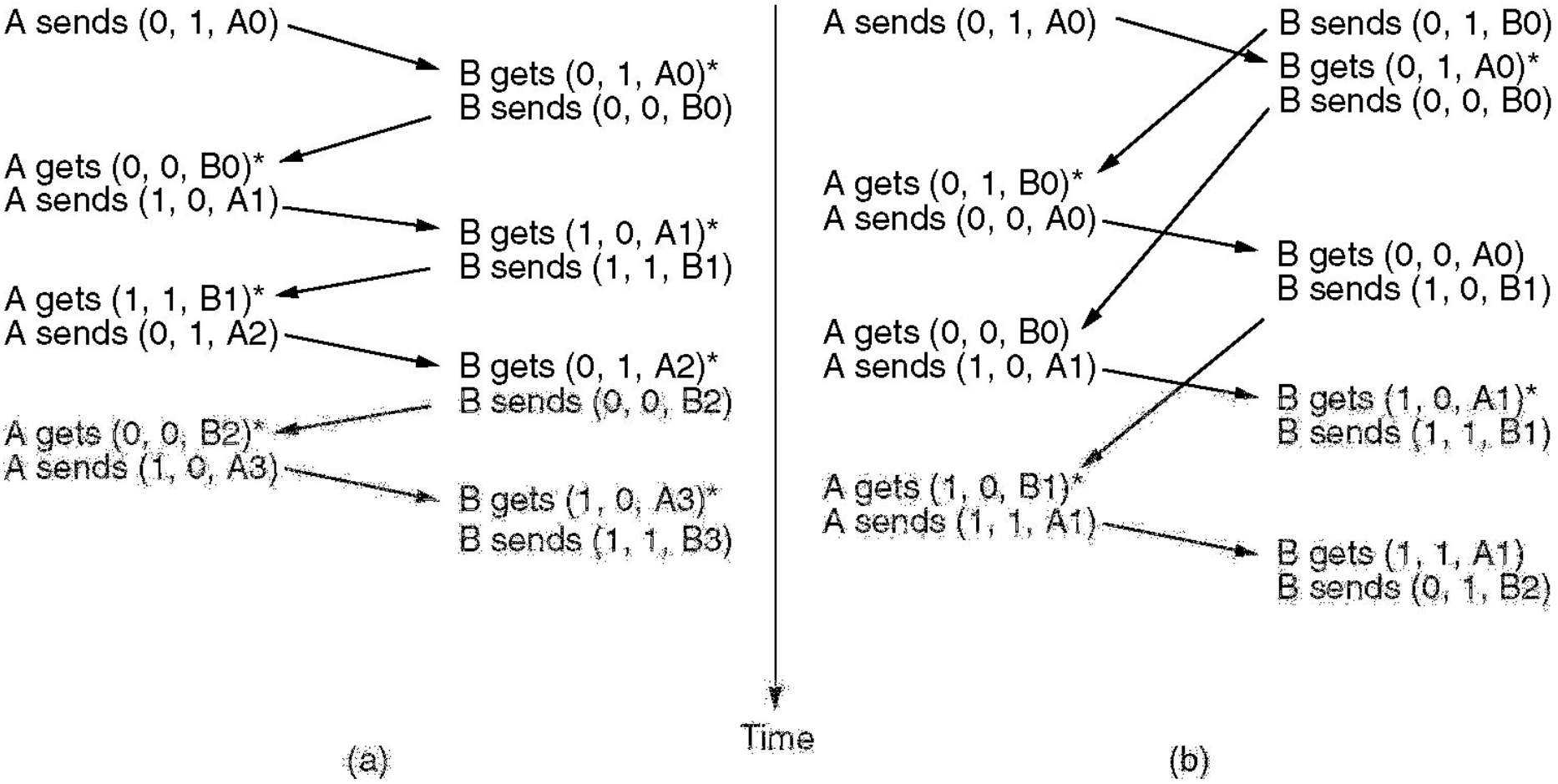
- **Configurations**
  - window size m = 1 (at both sender and receiver)
  - k = 1 ➔ maximum seq. # = 1➔possible seq.# = {0, 1}
  - Full duplex communication
    - Assume there is always data to Tx at both directions (direction 1: A -> B, direction 2: B -> A)
    - The Ack is piggybacked on data frames
    - Every host in network has the code for both sender and receiver.

SLIDING WINDOW PROTOCOL 1: 1-BIT WINDOW



SLIDING WINDOW PROTOCOL 1: 1-BIT WINDOW

Frame format: (seq#, Ack#, data)

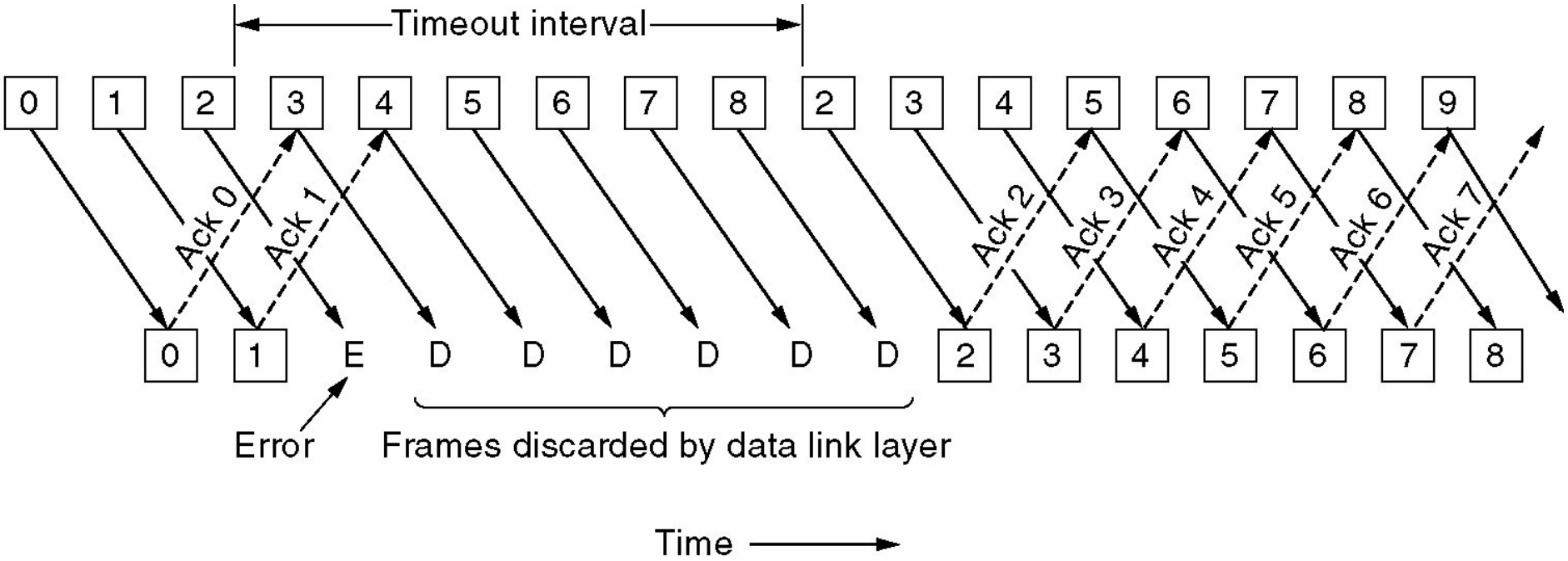


SLIDING WINDOW PROTOCOL 2: GO BACK N

- **Operations**
  - The sender window size is larger than 1 $1 \le m \le 2^k - 1$
    - Sender sends out several frames at one time (e.g. 0, 1, 2, 3, 4, 5)
  - What if error occurs to one frame in the middle? (e.g. 0, 1, 2, are correct, 3 is in error, 4, 5 are correct)
    - The receiver discards all the subsequent frames (3, 4, 5)
    - The receiver refuses to accept any frame except the next one in order (e.g. 3). ➔ the receiver will only accept one frame at one time ➔ the receiver window size is 1!
    - After time-out, the sender will re-Tx all the frames thereafter, (e.g. 3, 4, 5) ➔ Go Back N

SLIDING WINDOW PROTOCOL 2: GO BACK N

- **Example**



- **The receiver only accepts frame in order ➔ One frame at a time**
- **The sender needs to go back to the error frame and re-Tx all the frames thereafter ➔ go back N**

SLIDING WINDOW PROTOCOL 2: GO BACK N

- **Relationship between window size *m* and MAX_SEQ for Go Back N**
  - MAX_SEQ: maximum sequence number in the frame header
    - If *k* bits are used to represent the seq#, then

$$\text{MAX_SEQ} = 2^k - 1$$

    - [0, MAX_SEQ] can discriminate among MAX_SEQ+1 frames at one time.
  - For Go back N, window size m < MAX_SEQ+1. Why?
  - Example: k = 2, MAX_SEQ = 3, m = 4

[0 1 2 3] 0 1 2 3 0 1 2 3	(all the frames are received by B)
[0 1 2 3] 0 1 2 3 0 1 2 3	(A gets the Ack for frame 3)
Ack	
0 1 2 3 [0 1 2 3] 0 1 2 3	(A sends the next 4 frames in queue)
	(all frames lost in channel)
0 1 2 3 [0 1 2 3] 0 1 2 3	(B sends a data frame, with the Ack for the first frame 3 piggybacked in header)
Ack	
0 1 2 3 0 1 2 3 [0 1 2 3]	(A thinks the Ack is for the second frame 3)

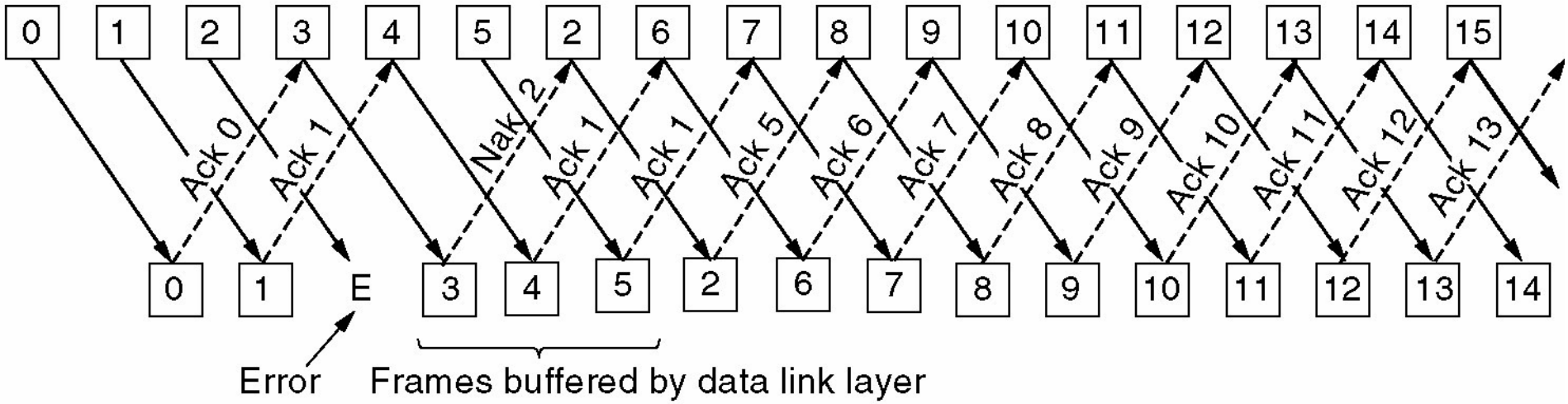
Window at sender (A)

SLIDING WINDOW PROTOCOL 3: SELECTIVE REPEAT

- **Operations**
  - Sender sends out several frames at one time (e.g. sends 0, 1, 2, 3, 4, 5)
  - At the receiver, bad frames are discarded. (e.g. 3 is in error, 3 is discarded, 0, 1, 2, 4, 5 are kept)
  - An Negative Ack (NAK) of the bad frame is sent back to transmitter
  - Good frames following the bad frames are kept in buffer (e.g. 4, 5 is going to be kept in buffer) ➔ receiver window is larger than 1
    - but Ack is sent out only for frame before the error frame (Ack is sent back for 2).
    - Receiver will only pass the frames to network layer in order
    - Out of order frames will be kept in buffer ➔ receiver needs to have buffer!
  - Sender will only re-Tx the frame with NAK, or time-out frame with the smallest seq#.➔ selective repeat

SLIDING WINDOW PROTOCOL 3: SELECTIVE REPEAT

- **Examples**



SLIDING WINDOW PROTOCOL 3: SELECTIVE REPEAT

- **Relationship between window size *m* and MAX_SEQ for Selective Repeat**
  - MAX_SEQ $= 2^k - 1$, k is the number of bits used to represent sequence number.
  - Example 1: k = 3, MAX_SEQ = 7, m = 7

	Host A (sender)	Host B (receiver)
A sends 7 frames	[0 1 2 3 4 5 6] 7 0 1 2 3 4 5 6 7	[0 1 2 3 4 5 6] 7
B send Acks	[0 1 2 3 4 5 6] 7 0 1 2 3 4 5 6 7	[0 1 2 3 4 5] 6 [7]
B advances its window		
All Acks are lost	[0 1 2 3 4 5 6] 7 0 1 2 3 4 5 6 7	[0 1 2 3 4 5] 6 [7]
A re-Tx 0		

B will accept frame 0 as a new frame!

  - Example 2: k = 3, MAX_SEQ = 7, m = 4

	Host A (sender)	Host B (receiver)
A sends 4 frames	[0 1 2 3] 4 5 6 7 0 1 2 3 4 5 6 7	[0 1 2 3] 4 5 6 7
B send Acks	[0 1 2 3] 4 5 6 7 0 1 2 3 4 5 6 7	0 1 2 3 [4 5 6 7]
B advances its window		
All Acks are lost	[0 1 2 3] 4 5 6 7 0 1 2 3 4 5 6 7	0 1 2 3 [4 5 6 7]
A re-Tx 0		

B will discard frame 0!

SLIDING WINDOW PROTOCOL 3: SELECTIVE REPEAT

- **Relationship between window size *m* and MAX_SEQ for Selective Repeat (Cont'd)**
  - To avoid the problem as described in example 1, there should be no overlap when the receiver advances its window.

$$m \leq (\text{MAX_SEQ} + 1)/2$$

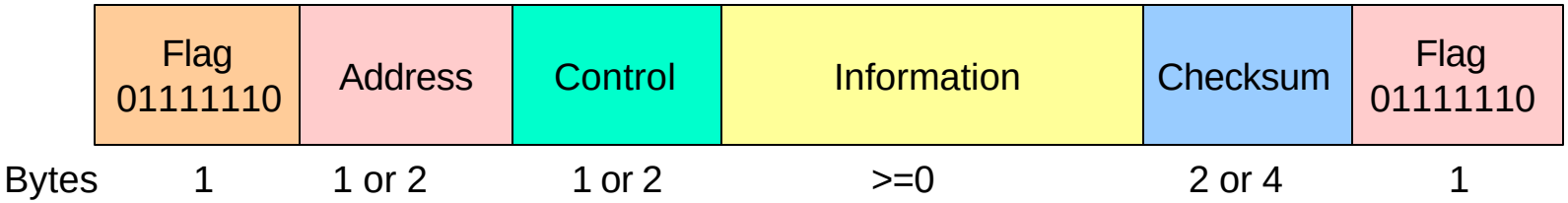
  - The sender needs to keep a copy of all the frames not acknowledged
    - Sender buffer size = sender window size
  - Receiver needs to handle out of order frames at receiver
    - Receiver buffer size = receiver window size

OUTLINE

- Functions and Design Issues of Data Link Layer
- Error Detection and Correction
- Elementary Data Link Protocols
- Sliding Window Protocols
- **Example Data Link Protocols**
  - HDLC (High-level Data Link Control)
  - PPP (Point to Point Protocol)

EXAMPLES: HDLC

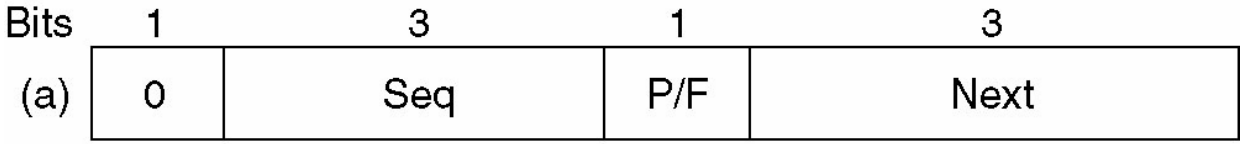
- **High-Level Data Link Control**
  - A bit-oriented connection-oriented protocol
    - Use flag with bit stuffing delimit frames
  - Has many varieties, still in use. (e.g. X.25)
- **Frame Structure**



  - Flag sequence: 01111110
  - Address: used to identify individual terminals on line with multiple terminals.
  - Control: seq. #, Ack, etc.
  - Checksum: cyclic redundancy code (CRC)
- **Error and flow control: sliding window**

EXAMPLES: HDLC CONTROL FIELD

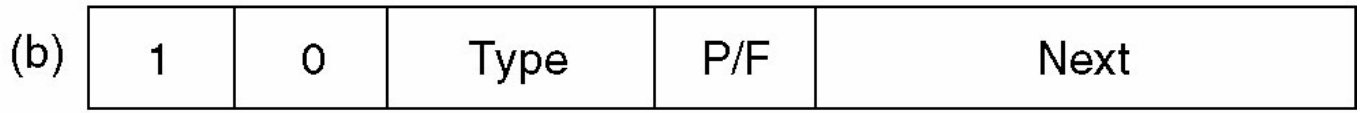
- **Three different kinds of frames**
  - Information frame
  - Supervisory frame
  - Unnumbered frame
- **Control field for information frame**



  - Seq: 3 bits of sequence number
  - Next: 3 bits used for piggyback acknowledgement
  - P/F: Poll/Final
    - When used as P, the computer is inviting the terminal to send data.

EXAMPLES: HDLC CONTROL FIELD

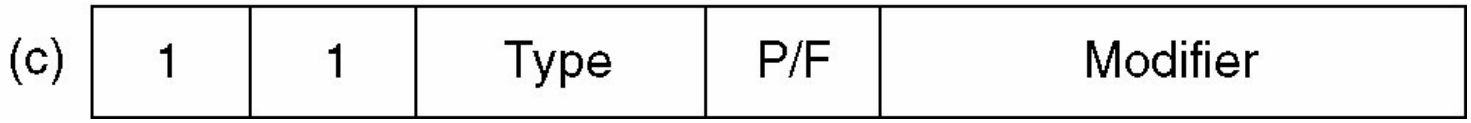
- **Control field for Supervisory frame**



  - Type 0: RECEIVE READY (Positive Acknowledgement)
    - Next field specifies the next frame expected
  - Type 1: REJECT (Negative Acknowledgement)
    - Next field specifies the first frame received in error
    - Sender needs to re-Tx all frames starting from the error frame.
      - Go Back N
  - Type 2: RECEICE NOT READY
    - Positive acknowledgement + tells sender stop sending
  - Type 3: SELECTIVE REJECT
    - Calls for re-Tx of the frame specified in Next
      - Selective Repeat

EXAMPLES: HDLC CONTROL FIELD

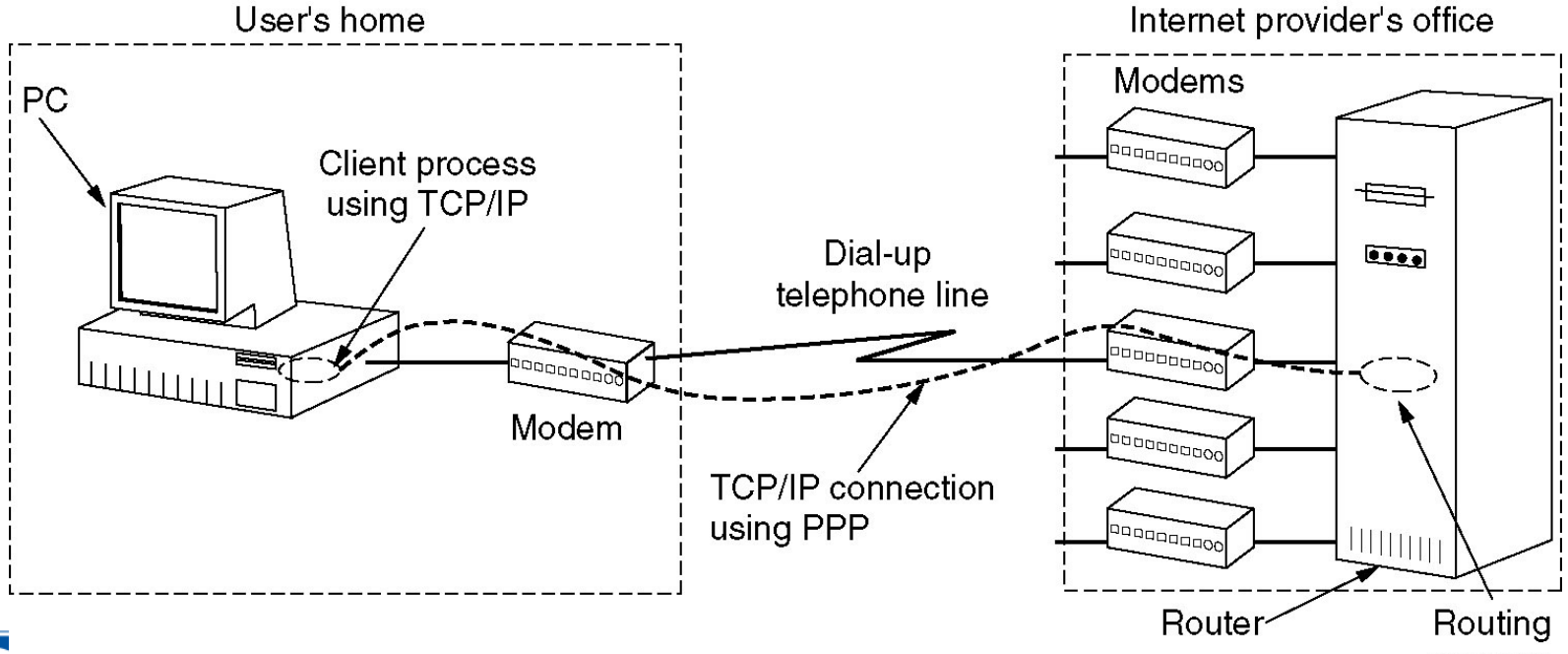
- **Control field for unnumbered frame**



  - Mainly used for control purpose
    - Disconnect, initialization, status reporting, etc.
  - Can also carry data for unreliable connectionless service

EXAMPLES: POINT-TO-POINT PROTOCOL (PPP)

- **Data link layer for Internet**
  - LANs use shared media (will be discussed in next chapter)
  - Routers are usually interconnected by point-to-point leased lines
  - Dial-up modem is using point-to-point link to connect to ISP.
    - Similar to leased line
  - PPP is used in Internet for data link layer with point-to-point link.

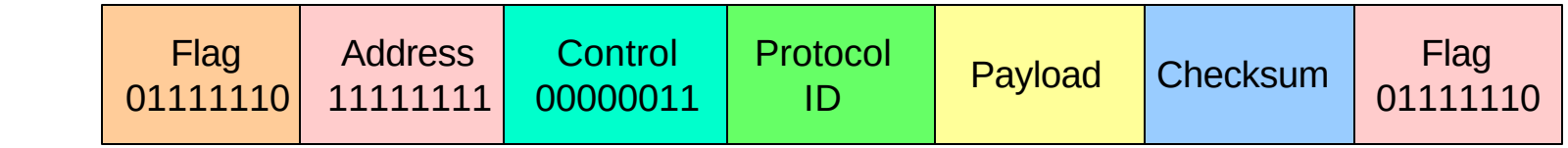


EXAMPLES: PPP

- **Features of PPP**
  - Framing: byte stuffing
  - Link Control Protocol (LCP)
    - Bringing lines up, negotiation options, bringing lines down
    - LCP packet is carried in the payload (data) field
  - Network Control Protocol (NCP)
    - Negotiate network layer options (e.g. IP address)
    - Different network layer protocol has different NCP
    - NCP packet is carried in the payload (data) field
  - Character oriented
    - frame length must be an integer number of bytes.
  - In general, a multiprotocol framing mechanism suitable for use over modems, SONET and other physical layers

EXAMPLES: PPP

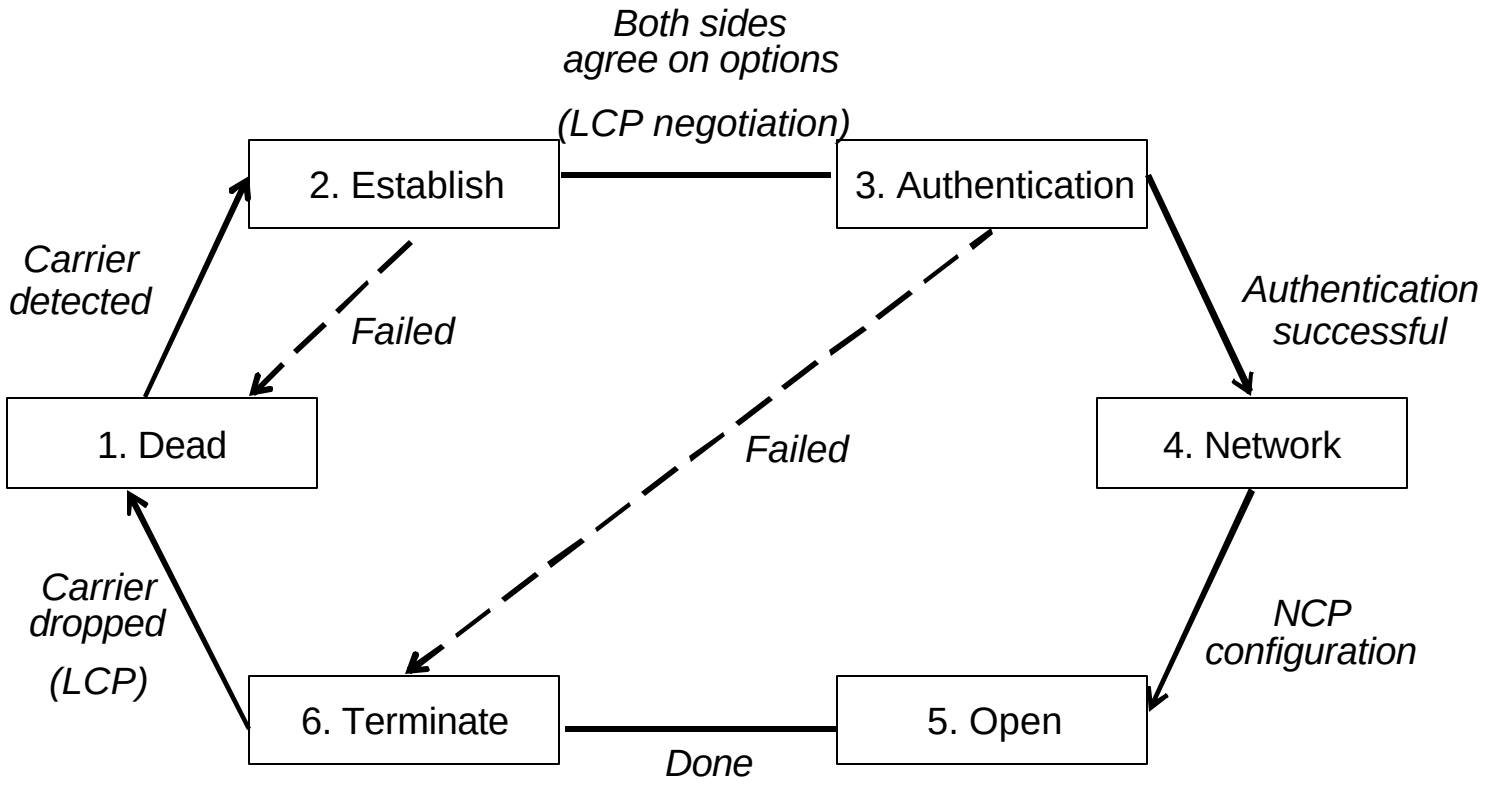
- **Frame Structure**



  - Flag: frame delimiter
  - Address: always set to 11111111, doesn't require address
  - Control: default value 00000011
    - Reliable Tx using sequence# and Ack is not default.
    - Can be set to other values to support reliable Tx.
    - Both address and control field can be omitted upon LCP negotiation.
  - Protocol: tells what kind of protocol is used in payload
    - IP, LCP, NCP, etc.
  - Payload: data, or protocol packet specified by Protocol field.
  - Checksum: error detection.

EXAMPLE: PPP

- **Operations**



SUMMARY

- **Functions of data link layer**
  - Provides reliable or unreliable communication between two adjacent hosts
  - Service to network layer, framing, error control, flow control
- **Framing**
  - Character count, Flag byte with byte stuffing, flag with bit stuffing, physical layer coding violation
- **Error Control**
  - Error correcting code, error detection code
- **Elementary data link protocols (simplex)**
  - Ideal, Stop-and-wait, ARQ
- **Sliding Window Protocols**
  - 1-bit sliding window, Go Back N, Selective Repeat
- **Examples**
  - HDLC, PPP